

What is `sqrt(2)`?

Programming and Tools for AI – Week 1

James McDermott



Part 1: Newton's algorithm

The problem: $\text{sqrt}(2)$

The calculator says 1.41421356...

Newton invented an algorithm for getting this answer, using only $+$, $-$, $*$, $/$.

A guess, and a better guess

Say we want $\text{sqrt}(16)$, and we guess 2.

- 2 is too small.
- So $16 / 2 = 8$ must be too large.

Two numbers which multiply to 16 as required, but one is too small, the other too large. So the answer is between them.

Average them: $(2 + 8) / 2 = 5$. This is an improvement!

Why it always works

If g is too small, then $16 / g$ is too large.

If g is too large, then $16 / g$ is too small.

Either way the answer lies between g and $16 / g$.

So their average is closer than at least one of them. Repeat.

Watch it converge

step	guess	16 / guess
0	2.0	8.0
1	5.0	3.2
2	4.1	3.902439024390244
3	4.001219512195122	3.998780859494057
4	4.0000001858445895	3.999999814155419
5	4.0000000000000004	3.99999999999999956

The code

```
def newton_sqrt(x, tol=1e-10):  
    if x < 0:  
        raise ValueError(f"Square root undefined for input {x}")  
    if x == 0:  
        return 0.0  
    g = float(x)  
    while True:  
        g = (g + x / g) / 2 # average  
        if abs(g * g - x) < tol: # no further progress  
            break  
    return g
```

Whitespace is structure

In some languages, indentation is decoration only, but in Python it defines structure.

```
while True:
    g = (g + x / g) / 2      # inside the loop
    if abs(g * g - x) < tol:
        break               # inside the if, inside the loop
return g                   # outside the loop
```

- A colon opens a **block**. The block is everything indented under it.
- Four spaces per level (be consistent).

The pieces

- `def name(args):` defines a function.
- `tol=1e-10` is a **default argument**: callers may omit it.
- `return` hands a value to the caller. A function usually returns rather than prints.
- `=` assigns. `g = (g + x / g) / 2` computes the right side, then assigns to `g`.
- `==` compares, ie asks a boolean question.
- `:` ends every line that opens a block: `def`, `if`, `while`, `for`.
- `while cond`: repeats while `cond` is true. `while True`: repeats forever.
- `break` leaves the innermost loop immediately.
- `abs(v)` is absolute value. We use it because the error can be either sign.
- `float(x)` converts to a floating-point number. `float(4)` is `4.0`.
- `raise` abandons the function with an error, instead of returning a wrong answer.

The substitution model

```
if abs(g * g - x) < tol:
```

Here `if` asks a question, which must be `True` or `False`. Python uses a **substitution model**, just like in maths and many other computer languages: **work inside-out**.

Let's suppose `g == 2`, `x == 16`, `tol=0.01`.

- `g * g` is 4
- `4 - 16` is -12
- `abs(-12)` is 12
- `12 < 0.01` is `False`

Pause

So far, we have a complete solution, but there are still several parts to be explained in the next section.

For now, try the first exercise, that is just to run `python -m doctest E01_newton.py` and read the code.

Part 2: more bits of Python

f-strings

Put an `f` before a string, and `{...}` interpolates a value:

```
>>> x = -1
>>> f"Square root undefined for input {x}"
'Square root undefined for input -1'
```

Any expression works inside the braces:

```
>>> f"{x} squared is {x * x}"
'-1 squared is 1'
```

Are we right?

```
>>> import math
>>> newton_sqrt(73.7)
8.584870412535581
>>> math.sqrt(73.7)
8.5848704125339
```

- `import math` makes the standard maths library available.
- `math.sqrt` is the name `sqrt` *inside* `math`.

The two answers are not identical. They differ by **floating-point error** `0.00000000000017`.

Doctests

An example in the docstring is documentation *and* a test.

```
def newton_sqrt(x, tol=1e-10):  
    """Return the square root of x.  
  
    >>> round(newton_sqrt(2), 6)  
    1.414214  
    >>> newton_sqrt(0)  
    0.0  
    """
```

Run them:

```
python -m doctest E02_first_last.py
```

No output means every test passed.

Scientific notation

$1\text{e}-10$ means 1×10^{-10} , that is 0.0000000001 .

- $1\text{e}3$ is 1000.0
- $2.5\text{e}-3$ is 0.0025
- Always a float, even $1\text{e}3$.

Easier to read than counting zeros.

Floats are not real numbers

A float has about 16 digits. Everything else is rounded, so we always have **floating-point error**.

```
>>> 0.1 + 0.2  
0.30000000000000004  
>>> 0.1 + 0.2 == 0.3  
False
```

So with floats we never ask “is it equal?”. We ask “is it close?”, using `tol`.

The shape of the algorithm

If we ignore the specific application (square roots), what is left is:

- a **starting point**
- a **transformation**
- a **stopping test**.

Many iterative algorithms use these parts, Newton's just one example.

Lego-like code

So, we could write the **shape** once, and pass in the parts:

```
def iterate(x, transform, done):  
    while not done(x):  
        x = transform(x)  
    return x
```

iterate is not specific to square roots.

Newton, rebuilt

```
def newton_sqrt(x, tol=1e-10):  
    if x < 0:  
        raise ValueError(f"Square root undefined for input {x}")  
    if x == 0:  
        return 0.0  
  
    def step(g):  
        return (g + x / g) / 2  
  
    def close_enough(g):  
        return abs(g * g - x) < tol  
  
    return iterate(float(x), step, close_enough)
```

- better and close_enough are **functions passed as arguments**.
- They are defined *inside* newton_sqrt, so they can see x and tol.

Part 3: A message we cannot read

An intercepted message

wkh txlfn eurzq ira

Every letter has been shifted along the alphabet by the same number of characters (**Caesar's cipher**). How many?

Letters are positions

```
>>> ALPHABET = "abcdefghijklmnopqrstuvwxyz"
>>> ALPHABET[3]
'd'
>>> ALPHABET.index('d')
3
```

Position to letter, and letter back to position.

Counting starts at **0**, so 'a' is at position 0 and 'z' is at 25.

Shifting one letter

Shift 'a' by 3: position $0 + 3 = 3$, which is 'd'.

Shift 'z' by 3: position $25 + 3 = 28$. There is no position 28.

Wrap around with %, the remainder operator:

```
>>> 28 % 26
```

```
2
```

```
>>> ALPHABET[2]
```

```
'c'
```

Shifting a message

```
def caesar(s, k):  
    out = []  
    for c in s:  
        if c in ALPHABET:  
            out.append(ALPHABET[(ALPHABET.index(c) + k) % 26])  
        else:  
            out.append(c)  
    return "".join(out)
```

Spaces and punctuation are not in ALPHABET, so they pass through unchanged.

Lists

- `[]` is an empty list. `out.append(x)` adds `x` to the end.
- A string cannot be changed in place. So we build a list, then join it.
- `"".join(out)` glues a list of strings into one string, with `""` between.
- `for c in s:` walks a string one character at a time.
- `c in ALPHABET` asks a question: is it there? Notice `in` has two slightly distinct uses.

A better idea

Instead of moving every letter, move the alphabet once:

```
>>> ALPHABET[3:] + ALPHABET[:3]  
'defghijklmnopqrstuvwxyzabc'
```

Now encrypting is just a lookup — no arithmetic at all:

```
shifted[ALPHABET.index(c)]
```

Slicing

- `s[2:5]` — from 2, up to but **not including** 5.
- `s[2:]` — from 2 to the end. `s[:5]` — the first five.
- `s[-1]` — the last item. `s[-3:]` — the last three.
- `s[::-1]` — reversed.
- Slicing a string gives a string; slicing a list gives a list.
- A slice is always a **new** object. The original is untouched.

We don't know k

So try all 26.

```
candidates = []  
for k in range(26):  
    candidates.append((k, caesar(s, -k)))
```

-k shifts backwards: if it was encoded with k, this decodes.

range(26) counts 0, 1, 2, ..., 25.

Tuples

(k, text) is a **tuple**:

- Round brackets. Once made, it cannot be changed.
- Unpack it in a loop: `for k, text in candidates:`
- Use a **tuple** when the parts mean different things, eg a number and a string.
- Use a **list** when the items are all the same kind of thing, and there may be any number of them.

Read down the list

```
0  wkh txlfn eurzq ira
1  vjg swkem dtqyp hqz
2  uif rvjdl cspxo gpy
3  the quick brown fox
4  sgd pthbj aqnvm enw
5  rfc osgai zpmul dmv
```

Could the computer choose?

English uses some letters far more than others: e, t, a, o, i, n.

For each of the 26 candidates, count the total occurrences of these letters, and call that the **score**.

Our **model** is then: pick the candidate of the 26 with the highest score.

We'll try this model in the exercises.

Summary

- Newton's algorithm: guess, improve, stop when close enough.
- Floats are not real numbers, so "equal" has to become "close enough".
- Breaking Caesar's cipher by brute force: generate all 26, then score them.
- The same shape twice: a starting point, a transformation, a stopping test. `iterate` is that shape, with the parts passed in as **functions**.

New this week: `float`, arithmetic and comparison operators, `if`, `while`, `for`, `break`, `def`, default arguments, `return`, `raise`; `abs`, `len`, `range`, `%`; f-strings and `print`; `import math`; `str`, `list` and `tuple`, indexing, slicing, `.index`, `.append`, `"".join`; functions passed as arguments. **Doctests**, and `python -m doctest`.